

HACKING



smart contracts

ethereum is not bitcoin



*“The key component
is this idea of a
Turing-complete
blockchain”*

--Vitalik Buterin

smart contracts



literally a billion reasons



Stephan Tual [Follow](#)

Slock.it Founder, Blockchain and Smart Contract Expert, Former CCO Ethereum

Jun 12 · 3 min read

No DAO funds at risk following the Ethereum smart contract ‘recursive call’ bug discovery

Our team is blessed to have Dr. Christian Reitwießner, Father of Solidity, as its Advisor. During the early development of the [DAO Framework 1.1](#) and thanks to his guidance we were made aware of a generic vulnerability common to all Ethereum smart contracts. We promptly circumvented this so-called “recursive call vulnerability” or “race to empty” from the DAO Framework 1.1 as can be seen on line [580](#):

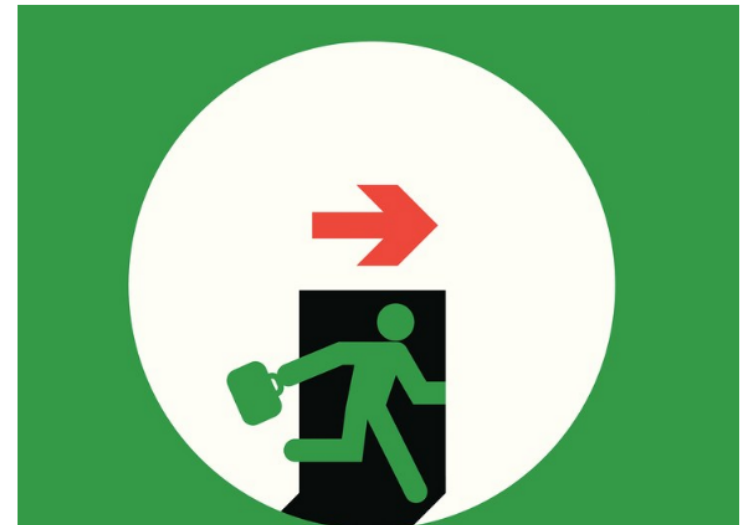
WIRED

A \$50 Million Hack Just Showed That the DAO Was All Too Human

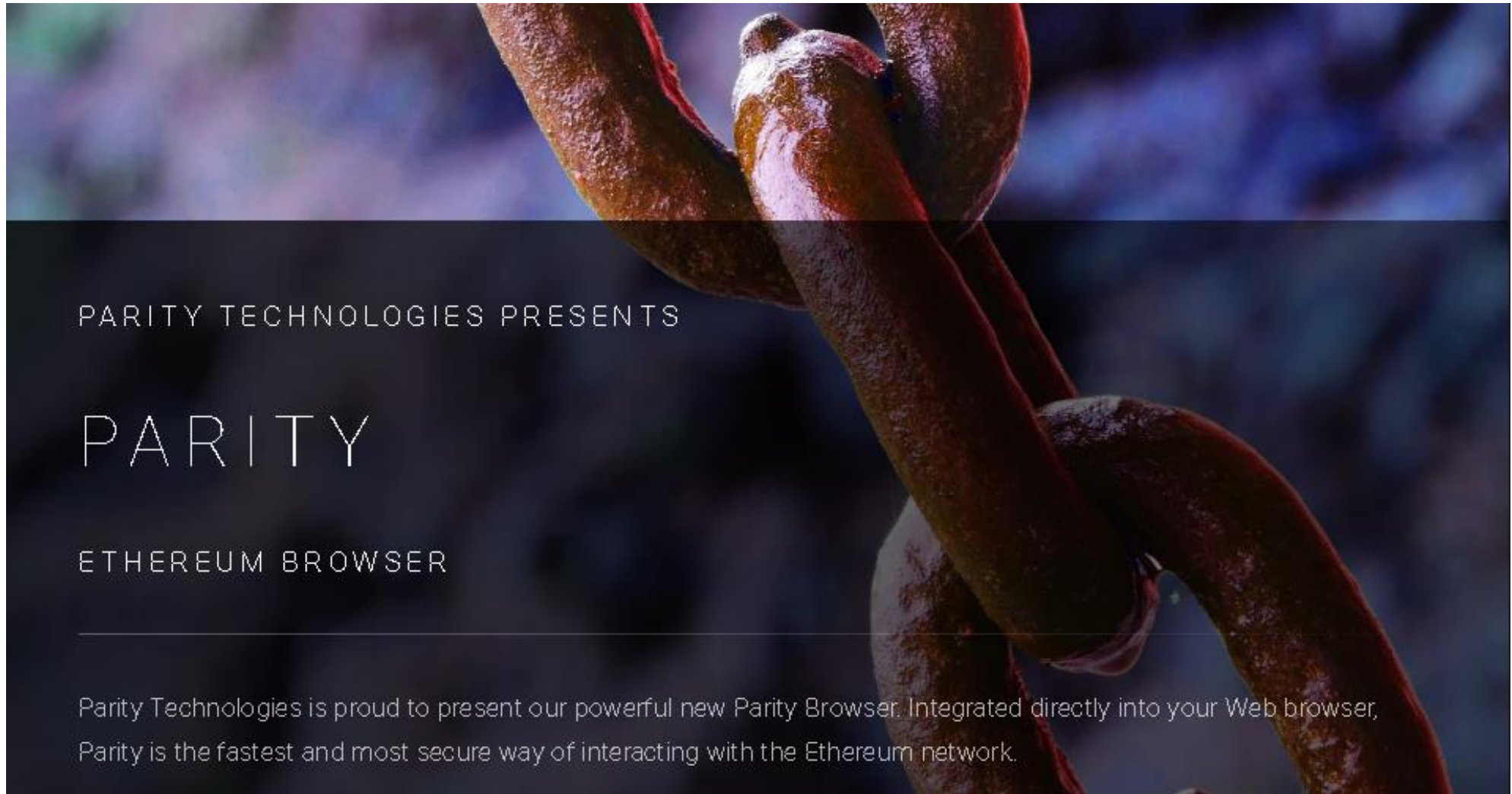
SHARE



A \$50 MILLION HACK JUST SHOWED THAT THE DAO WAS ALL TOO HUMAN



30 million reasons will do



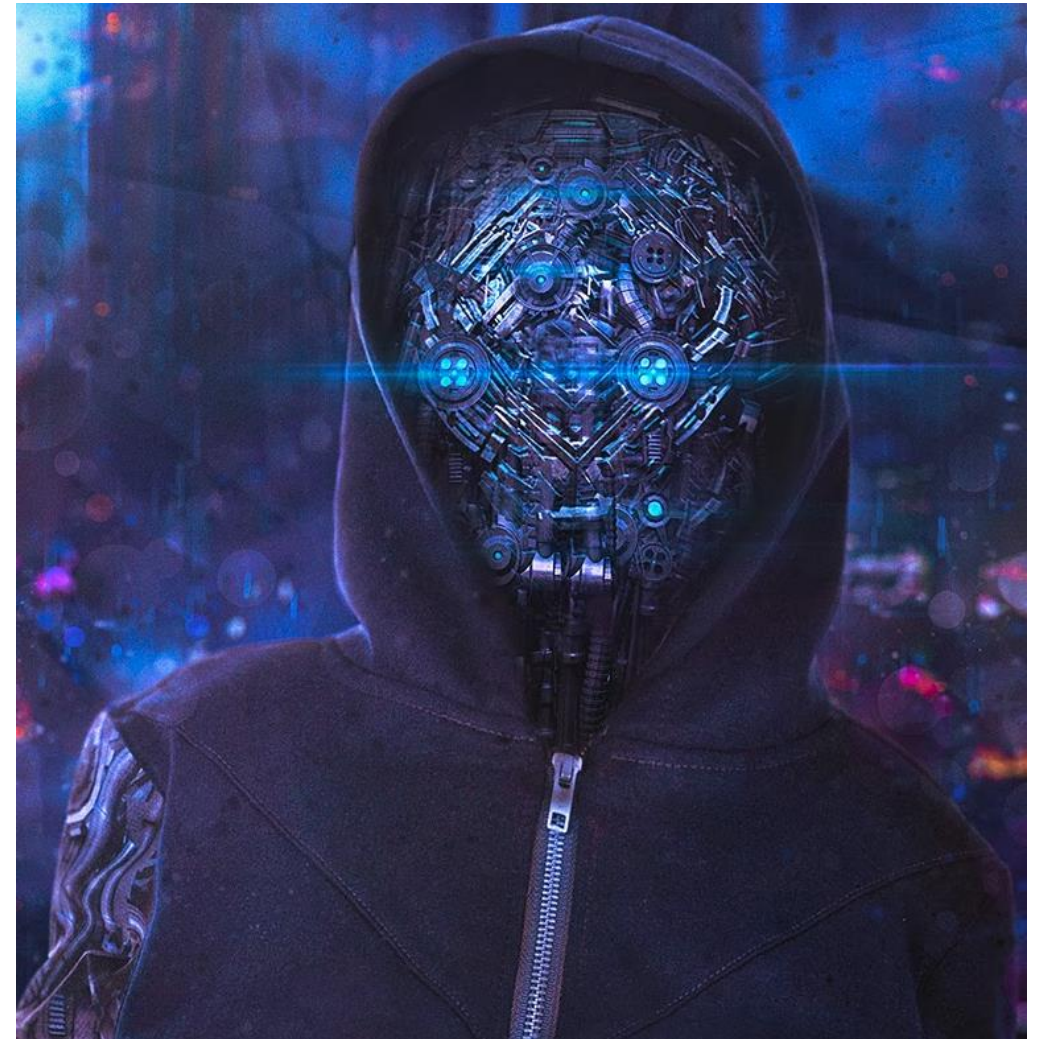
PARITY TECHNOLOGIES PRESENTS

PARITY

ETHEREUM BROWSER

Parity Technologies is proud to present our powerful new Parity Browser. Integrated directly into your Web browser, Parity is the fastest and most secure way of interacting with the Ethereum network.

caveats

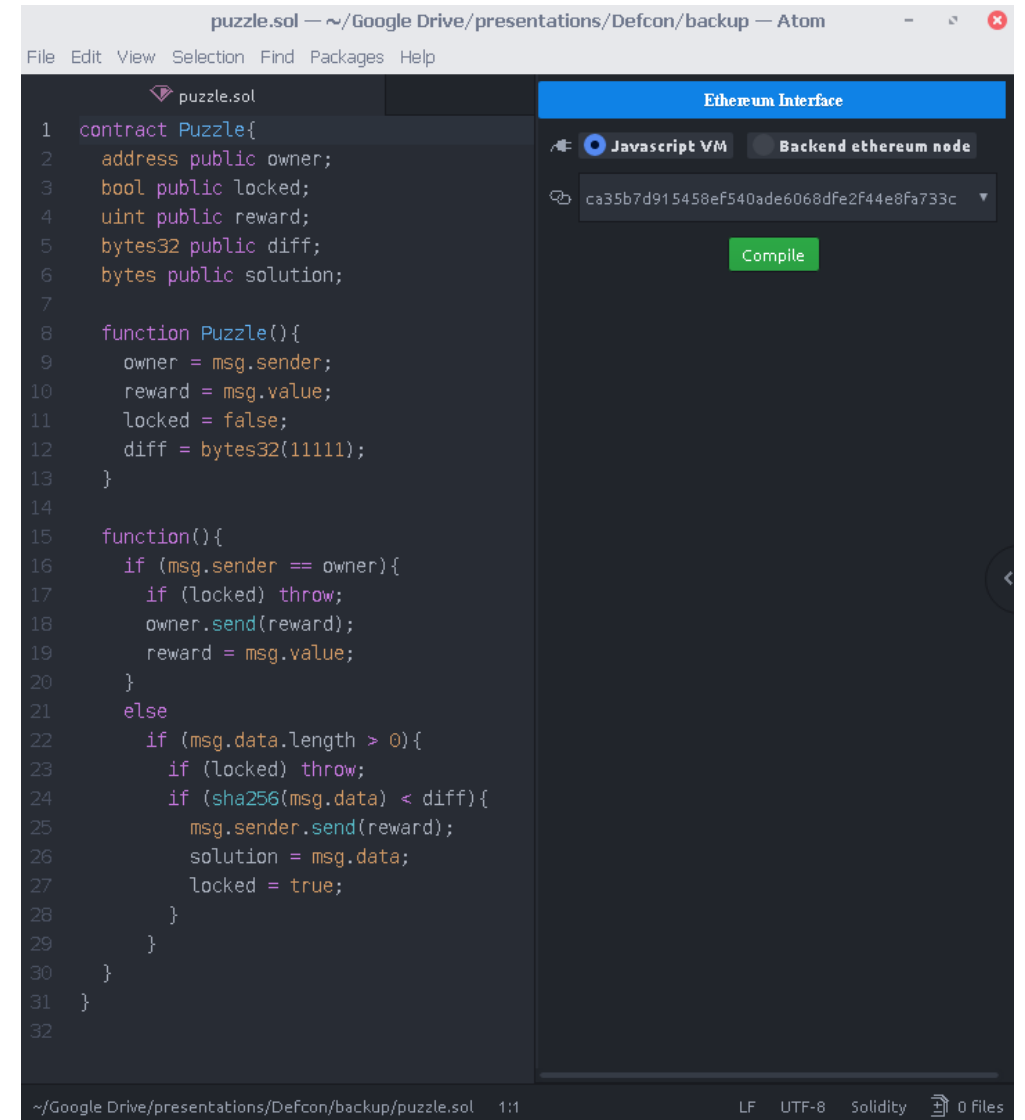


solidity



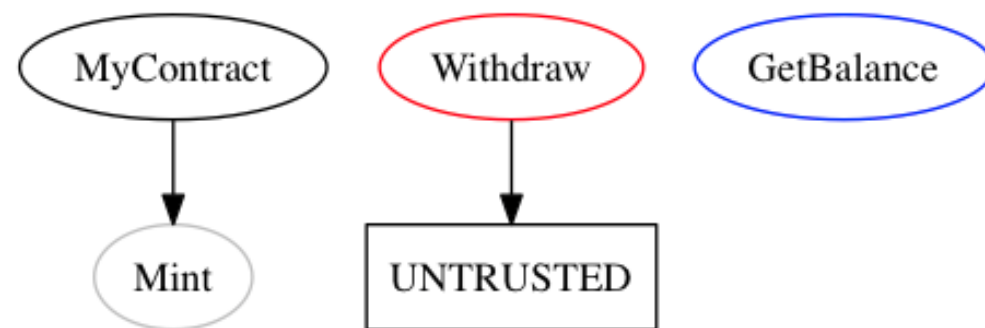
dev tools

- .sol files > bytecode > blockchain
- Atom with plugins:
 - language-ethereum
 - etheratom
- Remix: browser based



solgraph

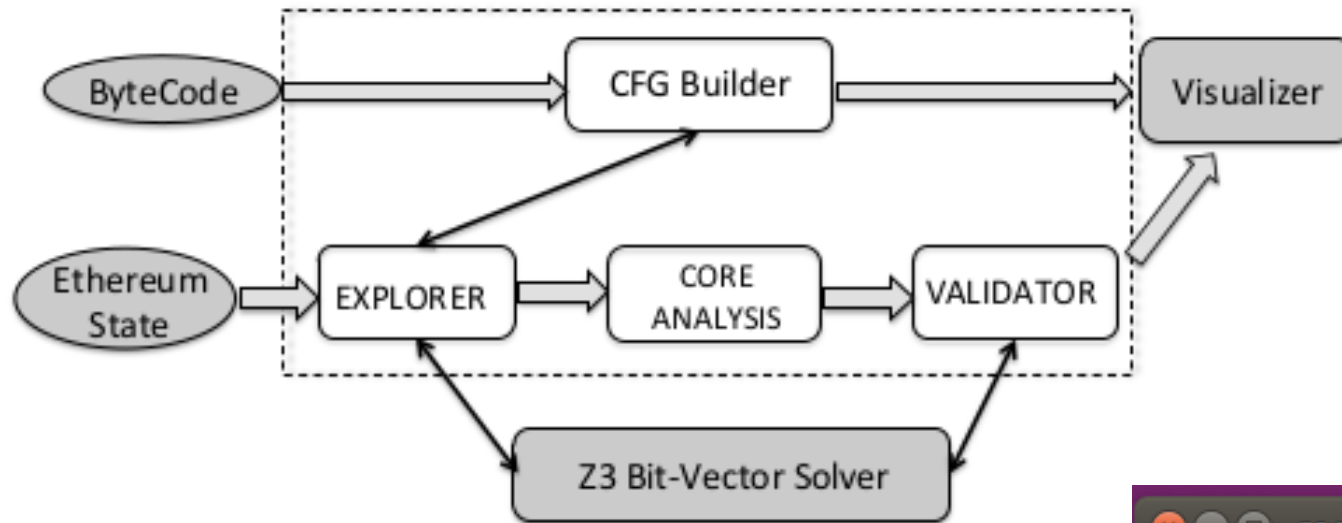
```
contract MyContract {  
  uint balance;  
  
  function MyContract() {  
    Mint(1000000);  
  }  
  
  function Mint(uint amount) internal {  
    balance = amount;  
  }  
  
  function Withdraw() {  
    msg.sender.send(balance);  
  }  
  
  function GetBalance() constant returns(uint) {  
    return balance;  
  }  
}
```



Legend:

- Black: Public function
- Gray: Internal function
- Red: Send to external address
- Blue: Constant function

oyente



```
root@f962f954fab1: /oyente
root@f962f954fab1:/oyente# python oyente.py -s defcon.sol
INFO:root:Contract defcon.sol:greeter:
INFO:symExec:Running, please wait...
INFO:symExec:  ===== Results =====
INFO:symExec:    CallStack Attack:      False
INFO:symExec:    Concurrency Bug:          False
INFO:symExec:    Time Dependency:          False
INFO:symExec:    Reentrancy bug exists: False
INFO:symExec:  ===== Analysis Completed =====
root@f962f954fab1:/oyente#
```

basic methodology

- Interview devs
- Review .sol file
- Try compiling
- Dissect code flow—optional solgraph
- Run oyente (cross fingers)
- Manually verify 3/4 vuln yay/nays
- Manually check for following vulns...



reentrancy



```
1  contract ReEntrancy {
2
3      mapping (address => uint) private expendableTokens;
4
5      function stealTokens() public {
6          uint amountToLose = expendableTokens[msg.sender];
7          if (!(msg.sender.call.value(amountToLose)())) { throw; }
8          expendableTokens[msg.sender] = 0;
9      }
10 }
```

leave off the first “re-” for savings



```
1  contract Entrancy {
2
3      mapping (address => uint) private expendableTokens;
4
5      function stealTokens() public {
6          uint amountToLose = expendableTokens[msg.sender];
7          expendableTokens[msg.sender] = 0;
8          if (!(msg.sender.call.value(amountToLose)())) { throw; }
9      }
10 }
```


reentrancy (and irony) in the dao code

```
// Burn DAO Tokens
    Transfer(msg.sender, 0, balances[msg.sender]);
    withdrawRewardFor(msg.sender); // be nice, and get his rewards
    totalSupply -= balances[msg.sender];
    balances[msg.sender] = 0;
    paidOut[msg.sender] = 0;
    return true;
}
```

default public – parity wallet hack

```
✠ @@ -104,7 +104,7 @@ contract WalletLibrary is WalletEvents {
104 104
105 105     // constructor is given number of sigs required to do protected "onlymanyowners" transactions
106 106     // as well as the selection of addresses capable of confirming them.
107 107 - function initMultiowned(address[] _owners, uint _required) {
107 107 + function initMultiowned(address[] _owners, uint _required) internal {
108 108     m_numOwners = _owners.length + 1;
109 109     m_owners[1] = uint(msg.sender);
110 110     m_ownerIndex[uint(msg.sender)] = 1;
✠ @@ -198,7 +198,7 @@ contract WalletLibrary is WalletEvents {
198 198     }
199 199
200 200     // constructor - stores initial daily limit and records the present day's index.
201 201 - function initDaylimit(uint _limit) {
201 201 + function initDaylimit(uint _limit) internal {
216 216 +
214 217     // constructor - just pass on the owner array to the multiowned and
215 218     // the limit to daylimit
216 216 - function initwallet(address[] _owners, uint _required, uint _daylimit) {
219 219 + function initwallet(address[] _owners, uint _required, uint _daylimit) only_uninitialized {
```

initWallet

Transaction Information		Tools & Utilities ▾
TxHash:	0x9dbf0326a03a2a3719c27be4fa69aacc9857fd231a8d9dcaede4bb083def75ec	
Block Height:	4043800 (28739 block confirmations)	
TimeStamp:	6 days 5 hrs ago (Jul-19-2017 12:18:15 PM +UTC)	
From:	0xb3764761e297d6f121e79c32a65829cd1ddb4d32 (MultisigExploit-Hacker)	
To:	Contract 0xbec591de75b8699a3ba52f073428822d0bfc0d7e 	
Value:	0 Ether (\$0.00)	
Gas Limit:	82703	
Gas Price:	0.000000021 Ether (21 Gwei)	
Gas Used By Txn:	66839	
Actual Tx Cost/Fee:	0.001403619 Ether (\$0.29)	
Cumulative Gas Used:	1283734	
Nonce:	5	
Input Data:	<pre>Function: initWallet(address[] _owners, uint256 _required, uint256 _daylimit) *** MethodID: 0xe46dcfeb [0]: 00 00000060 [1]: 00 00000000 [2]: 00 00000000</pre>	

execute

[illegible]

unchecked send in king of the ether



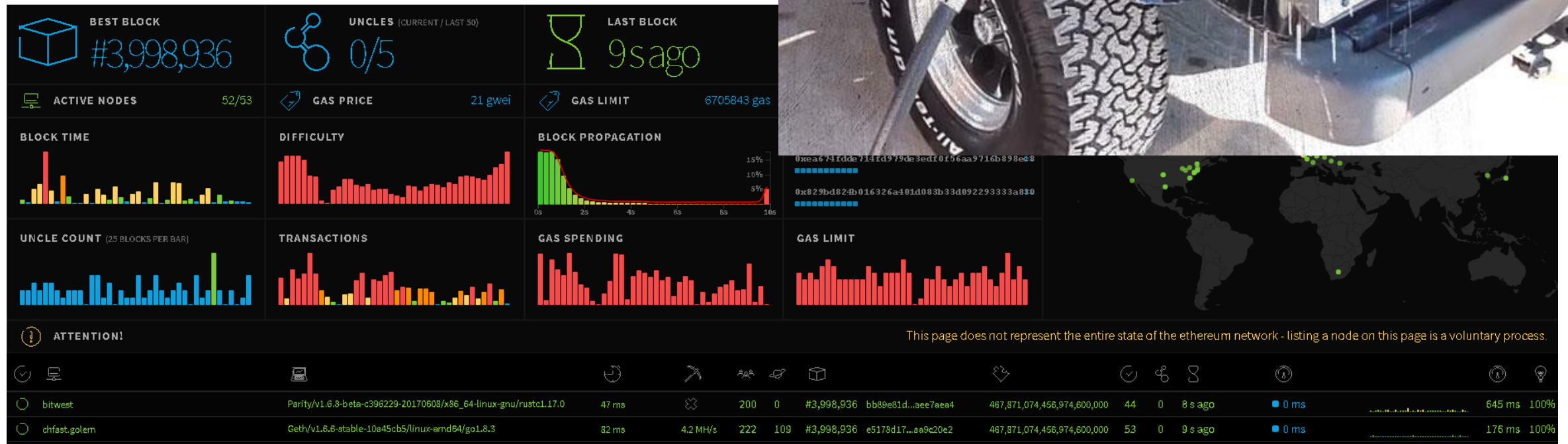
```
← → ↻ 🏠 🔒 GitHub, Inc. [US] | https://github.com/kieranelby/KingOfTheEtherThrone/blob/v0.4.0/contr
117     uint compensation = valuePaid - wizardCommission;
118
119     if (currentMonarch.etherAddress != wizardAddress) {
120         currentMonarch.etherAddress.send(compensation);
121     } else {
122         // When the throne is vacant, the fee accumulates for the wizard.
123     }
124
```

unchecked send

```
1  if (kingOfLosingDone && !( compensationSent ) ) {  
2      monarch.send(500);  
3      compensationSent = True;  
4  }
```

```
1  if (kingOfLosingDone && !( compensationSent ) ) {  
2      if (monarch.send(500))  
3          compensationSent = True;  
4      else throw;  
5  }
```

gas limits



withdraw don't send



```
1  contract SendContract {
2      address public richest;
3      uint public mostSent;
4
5      function SendContract() payable {
6          richest = msg.sender;
7          mostSent = msg.value;
8      }
9
10     function becomeRichest() payable returns (bool) {
11         if (msg.value > mostSent) {
12             richest.transfer(msg.value);
13             richest = msg.sender;
14             mostSent = msg.value;
15             return true;
16         } else {
17             return false;
```

withdrawn not sent

```
1  contract WithdrawalContract {
2      address public richest;
3      uint public mostSent;
4
5      mapping (address => uint) pendingWithdrawals;
6
7      function WithdrawalContract() payable {
8          richest = msg.sender;
9          mostSent = msg.value;
10     }
11
```

```
12     function becomeRichest() payable returns (bool) {
13         if (msg.value > mostSent) {
14             pendingWithdrawals[richest] += msg.value;
15             richest = msg.sender;
16             mostSent = msg.value;
17             return true;
18         } else {
19             return false;
20         }
21     }
22
23     function withdraw() {
24         uint amount = pendingWithdrawals[msg.sender];
25         pendingWithdrawals[msg.sender] = 0;
26         msg.sender.transfer(amount);

```

encryption



transaction-ordering dependence

```
1  contract Puzzle{
2      address public owner;
3      bool public locked;
4      uint public reward;
5      bytes32 public diff;
6      bytes public solution;
7
8      function Puzzle(){
9          owner = msg.sender;
10         reward = msg.value;
11         locked = false;
12         diff = bytes32(11111);
13     }
14
```



```
15  function(){
16      if (msg.sender == owner){
17          if (locked) throw;
18          owner.send(reward);
19          reward = msg.value;
20      }
21      else
22          if (msg.data.length > 0){
23              if (locked) throw;
24              if (sha256(msg.data) < diff){
25                  msg.sender.send(reward);
26                  solution = msg.data;
27                  locked = true;
28              }
29          }
30      }
31  }
```

call-stack depth limit

```
root@f962f954fab1: /oyente
root@f962f954fab1:/oyente# python oyente.py -s defcon.sol
INFO:root:Contract defcon.sol:greeter:
INFO:symExec:Running, please wait...
INFO:symExec:  ===== Results =====
INFO:symExec:  Callstack Attack:      False
INFO:symExec:  Concurrency Bug:         False
INFO:symExec:  Time Dependency:              F
INFO:symExec:  Reentrancy bug exists: F
INFO:symExec:  ===== Analysis Completed =====
root@f962f954fab1:/oyente#
```



Ethereum 
@ethereumproject

Following



Announcement of imminent hard fork for EIP150 gas cost changes:



Announcement of imminent hard fork for EIP150 gas cost...

During the last couple of weeks, the Ethereum network has been the target of a sustained attack. The attacker(s) have been very crafty in locating vulnerabilities in the client implementations as...

blog.ethereum.org

5:28 PM - 13 Oct 2016

variable or function ambiguity

```
1  Player[] public persons;
2
3  uint public payoutCursor_Id_ = 0;
4  uint public balance = 0;
5
6  address public owner;
7
8  uint public payoutCursor_Id=0;
9  ...
10 while (balance > persons[payoutCursor_Id_].deposit / 100 * 115) {
11     uint MultipliedPayout = persons[payoutCursor_Id_].deposit / 100 * 115;
12     persons[payoutCursor_Id].etherAddress.send(MultipliedPayout);
13
14     balance -= MultipliedPayout;
15     payoutCursor_Id_++;
```

input validation

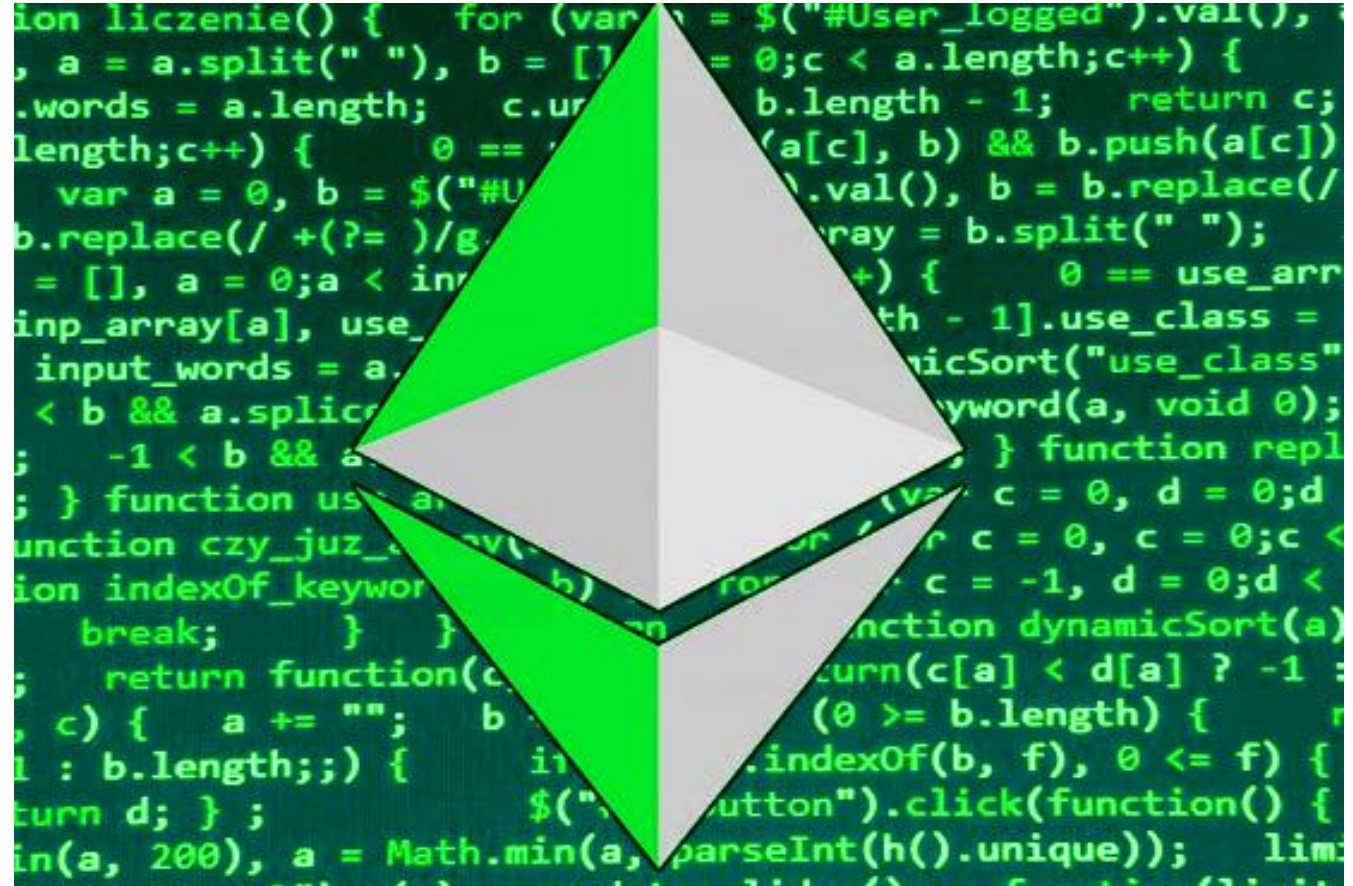
throw -- being deprecated

require (condition) – check external conditions
(invalid inputs or errors in external components)

assert (condition) – internal errors

odds and ends

- Timestamp dependence
- Business logic flaws
- Separating public/private data



get involved



dox me ... or just keep in touch

@konstanthacker

konstantinos.karagiannis@bt.com

